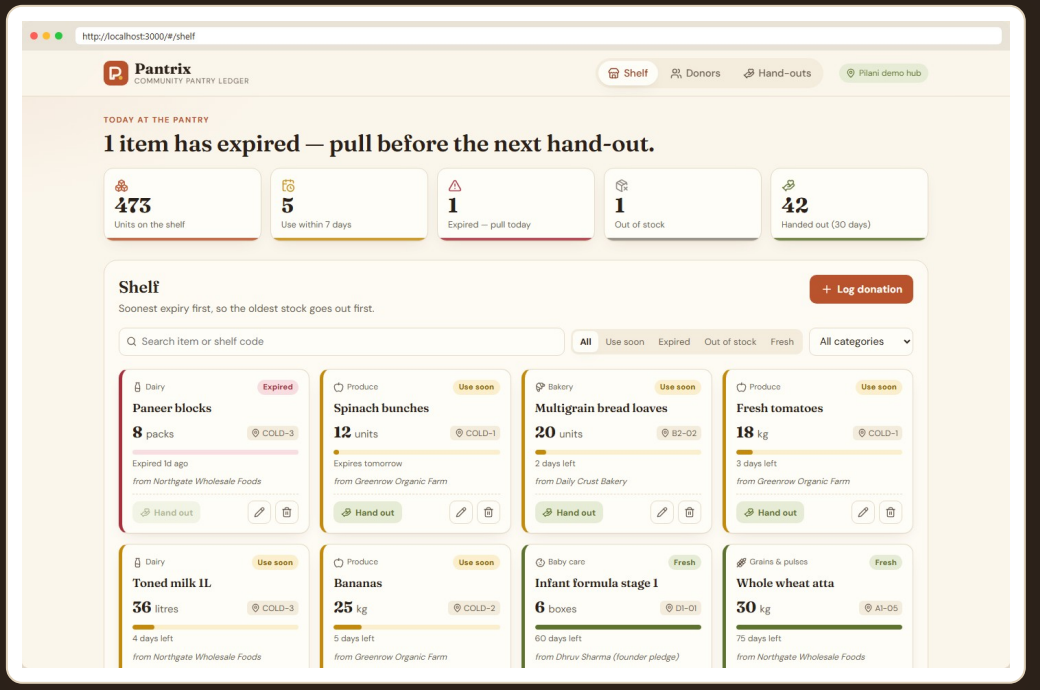
Pantrix

Donated stock and expiry tracking for community food banks, taken from a laptop to a monitored Kubernetes cluster

DevSecOps capstone · Dhruv Sharma · @Spiritsfuse · github.com/spiritsfuse



Food banks lose donated food to expiry, not to demand

Know what expires first

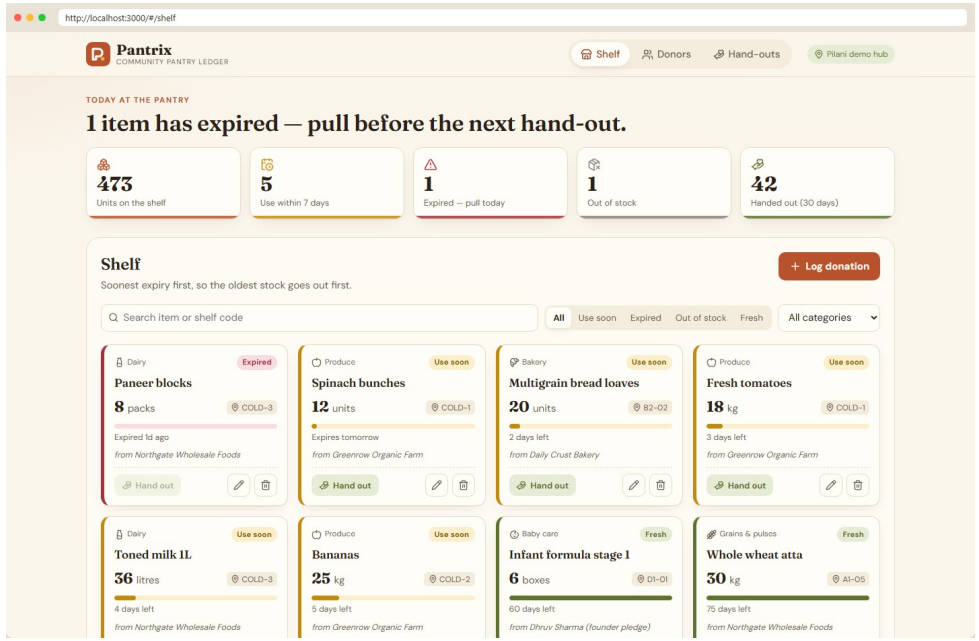
Every item has an expiry date; the shelf is sorted soonest-first and flagged Use soon (7 days), Expired or Out of stock.

Record every hand-out

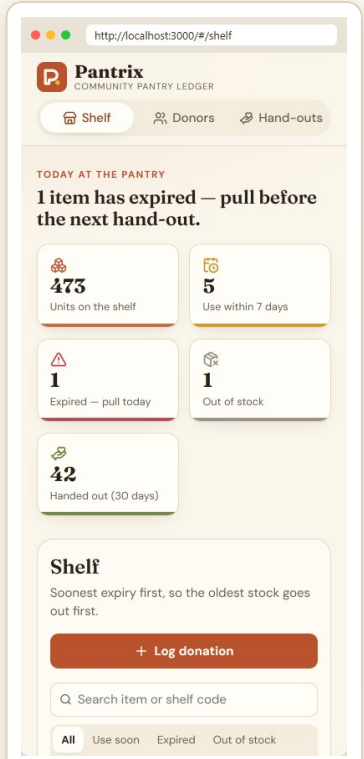
Distributions decrement stock in one transaction; expired or over-asked stock is refused with HTTP 409.

Credit donors

Farms, kitchens and neighbours are tracked with the items they gave.

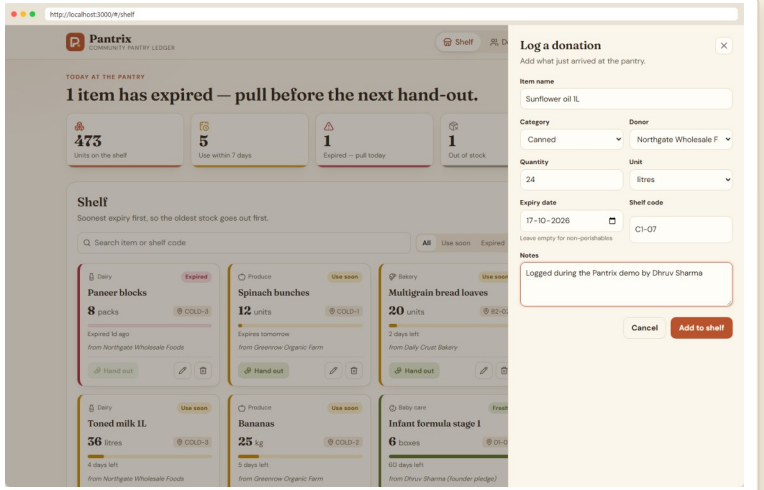


Shelf view served by Docker Compose (localhost:3000)

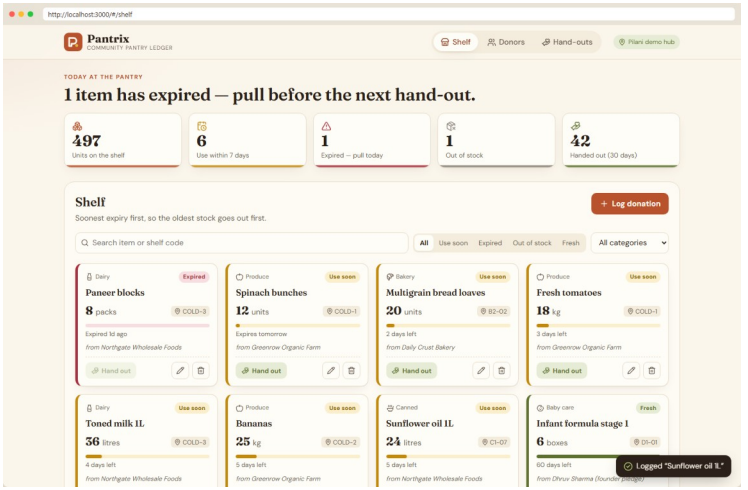


390 px mobile layout

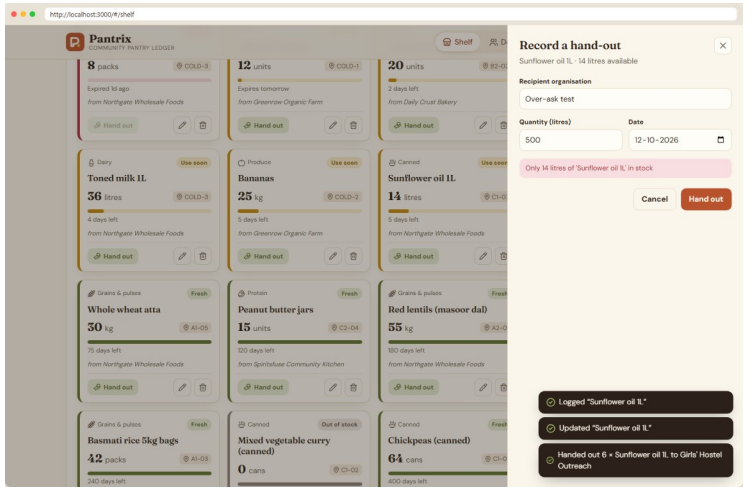
Real CRUD through the API, validation users can see



Create: log a donation in the slide-in form



Created: toast, KPI strip and shelf update



API rule: cannot hand out more than in stock (409)

- React 19 + Vite frontend calls relative /api paths, so the same build works behind Vite, Nginx and the Ingress
- Every screen has loading skeletons, empty states, error banners with retry, and success toasts
- Captured with Playwright against the live stack: create, edit, hand out, API conflict, delete (docs/evidence/crud-*.png)

Tech stack

Application

React 19 · Vite 8 · FastAPI · SQLAlchemy 2 · Alembic · PostgreSQL 17 · Pydantic validation

Quality gates

pytest + coverage · Vitest + Testing Library · Ruff · ESLint · Prettier · Bandit · pip-audit · npm audit

Containers

Multi-stage Dockerfiles · non-root users · nginx-unprivileged · Docker Compose with health checks

CI/CD and security

GitHub Actions · Trivy (images + IaC) · Gitleaks · CycloneDX SBOM · GHCR with commit-SHA tags · provenance

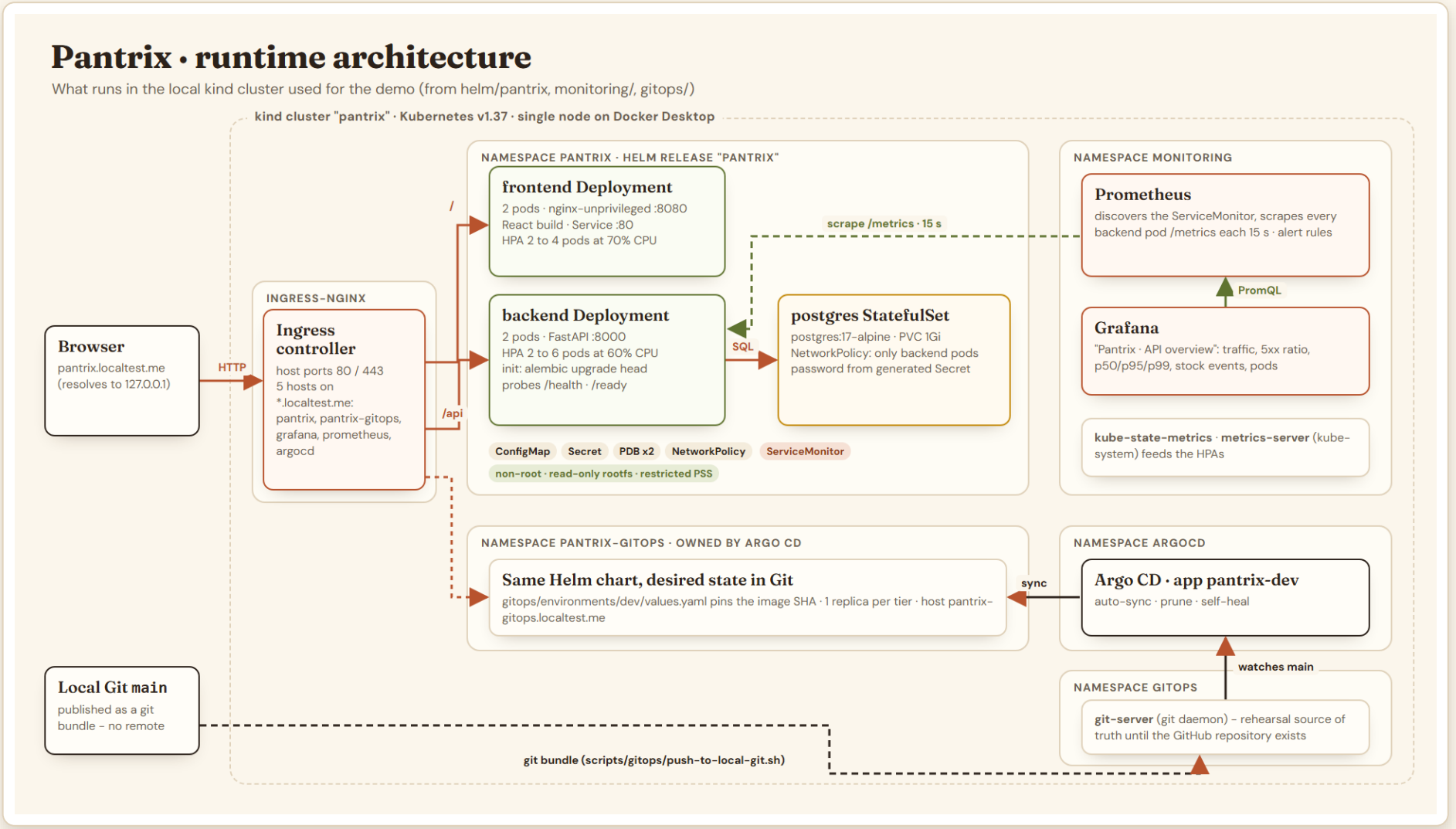
Platform

Terraform (AWS VPC + EKS) · kind · Helm · ingress-nginx · HPA + metrics-server · NetworkPolicy

Operations

Argo CD (GitOps) · Prometheus Operator · Grafana · JSON logs · five troubleshooting drills

Runtime architecture on the local kind cluster



Data model and REST API

donors

name (unique) · donor_type · contact_email · city

stock_items

name · category · quantity >= 0 · unit · expiry_date · location · donor_id (SET NULL)

distributions

item_id · quantity > 0 · recipient · distributed_on; creating one decrements stock under a row lock

Method	Path	Purpose
GET	/api/items	list; filter by category, status, search
GET	/api/items/stats	KPIs: units, expiring, expired, hand-outs
GET · PUT · DELETE	/api/items/{id}	read, partial update, remove (404 if missing)
POST	/api/items	create, 201; 422 on invalid input
GET · POST	/api/donors	list with item counts; create (409 on duplicate)
GET · PUT · DELETE	/api/donors/{id}	read, update, delete (items kept)
GET · POST	/api/distributions	history; hand out (409 if short or expired)
GET	/health · /ready · /metrics	liveness, DB readiness, Prometheus

Created by Alembic migration 0001_initial_schema

Tests are the first gate: nothing builds until they pass

31

backend tests, pytest -v

96%

API line coverage (CI floor: 85%)

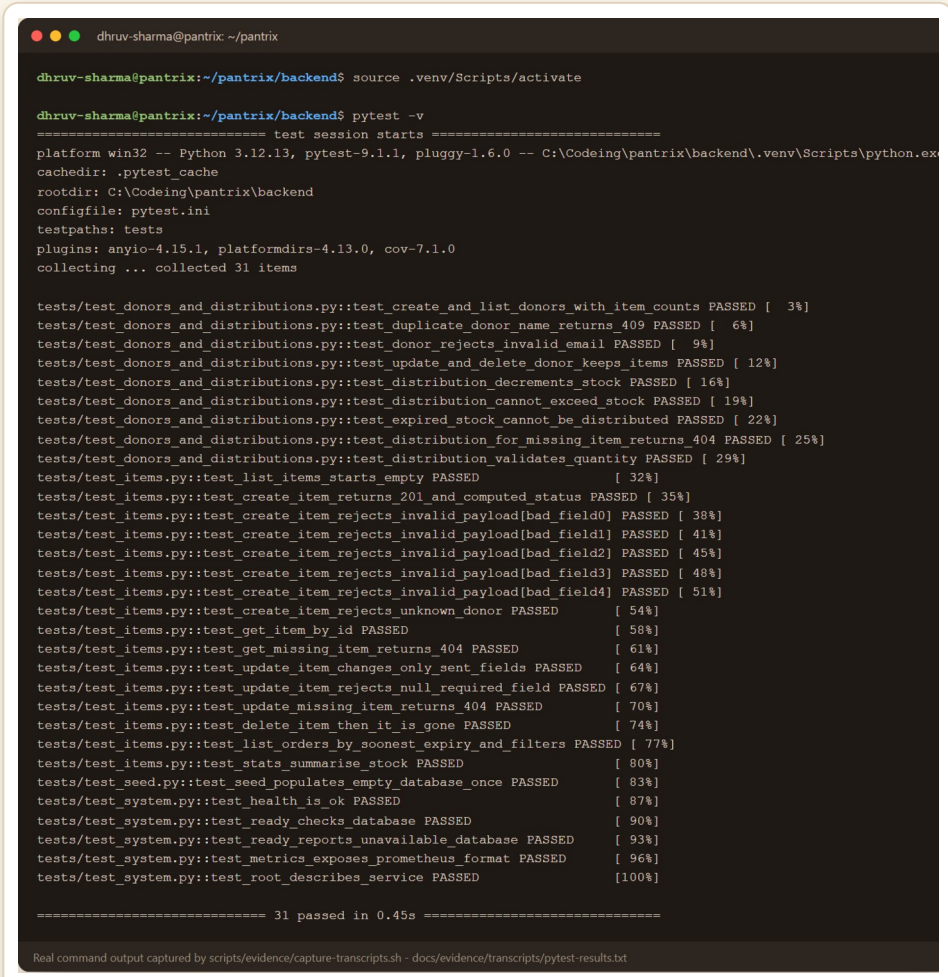
10

frontend tests with Vitest

0

findings: Ruff, Bandit, pip-audit, npm audit

- Tests run against a throw-away SQLite database built by the real Alembic migrations, never the Compose or cluster PostgreSQL
- Cover success, validation (422), missing records (404), conflicts (409), readiness failure and the /metrics format



pytest -v: 31 passed (real output)

Containers: multi-stage, non-root, one command to run

uid 10001

backend runs as pantrix

uid 101

frontend runs as nginx

311 / 114 MB

backend / frontend image size

- Backend: build stage resolves a virtualenv; runtime has no pip, no compiler, migrates then serves
- Frontend: Node build stage, then nginx-unprivileged on 8080 proxying /api
- docker compose up --build: PostgreSQL, API and UI with health checks, a named volume and dropped capabilities

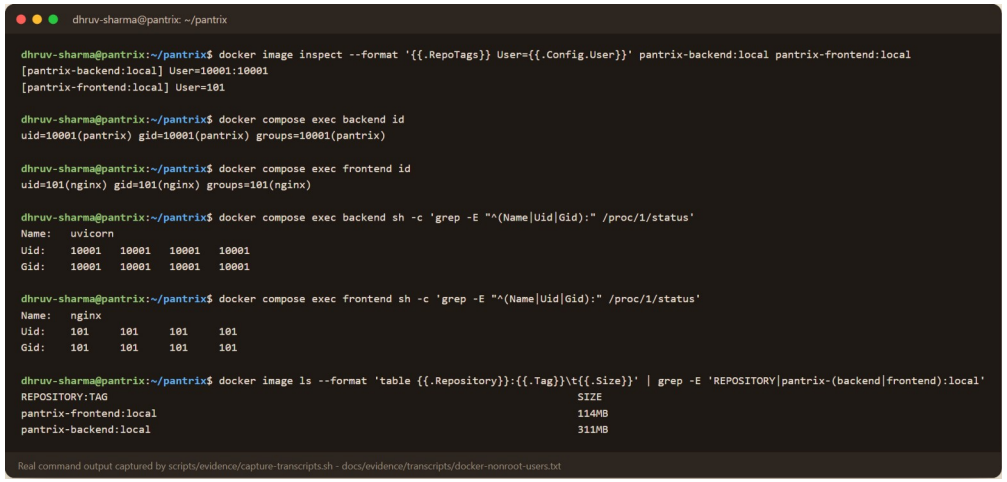
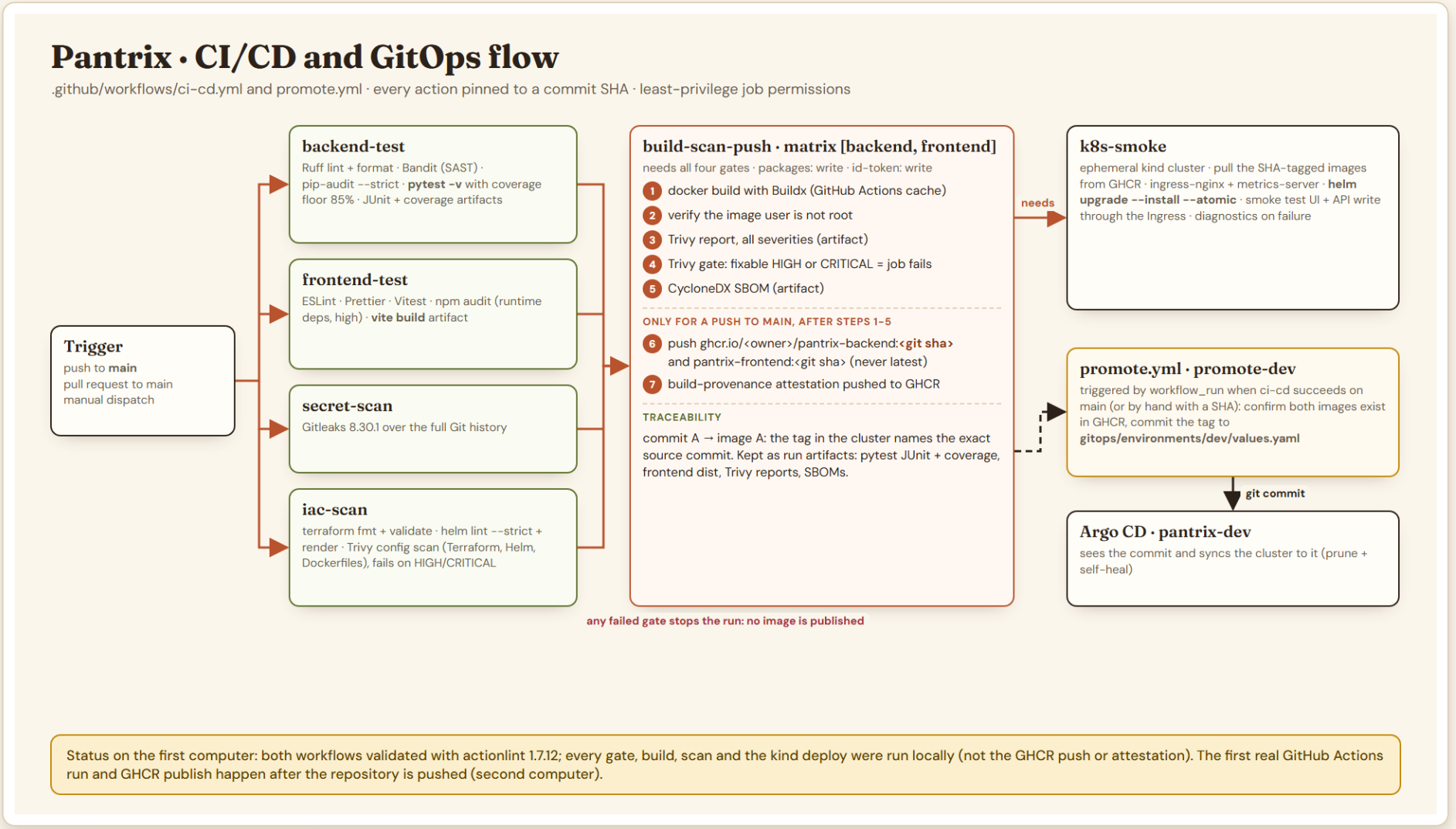


Image user and PID 1 user inside both containers

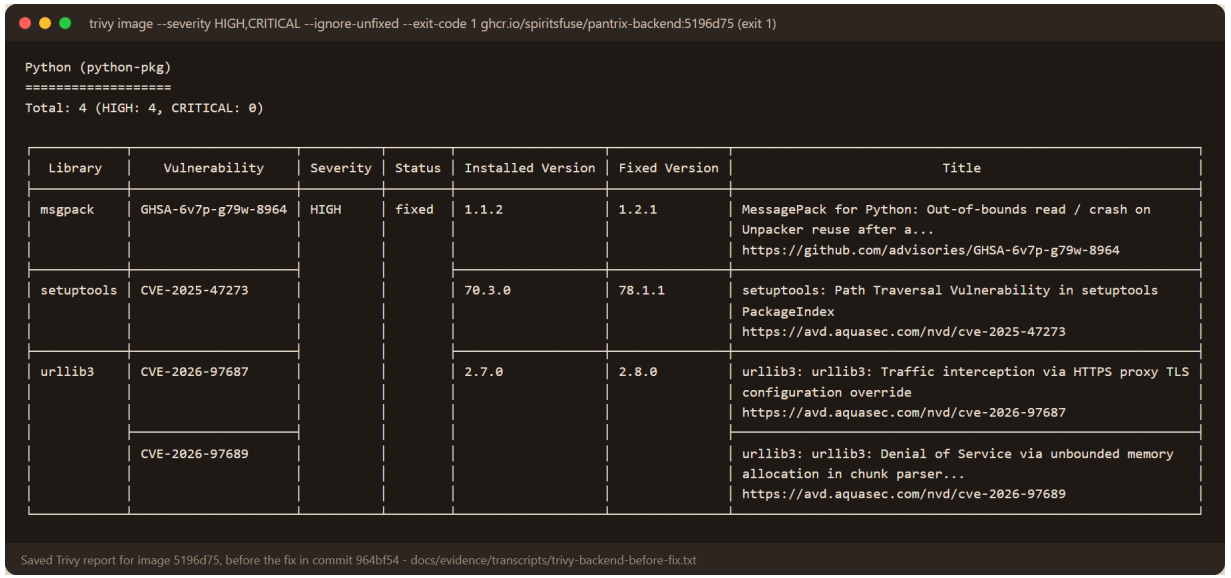


All three Compose services healthy

CI/CD: gates first, then build, scan, publish by SHA



DevSecOps: Trivy found real issues, and we fixed them



Before: image 5196d75 failed the gate with 4 fixable HIGH CVEs

- Findings came from pip's vendored urllib3 and msgpack and the ensurepip setuptools wheel, which the API never imports
- Fix in commit 964bf54: remove pip from the runtime image. Gate now passes; nothing suppressed
- Policy: fail on fixable HIGH or CRITICAL. Unfixed Debian CVEs are reported, not blocking

Infrastructure scan

Trivy flagged a public EKS API open to 0.0.0.0/0 (AWS-0040/0041): now private unless you pass your own CIDR. AWS-0104 (node egress) is a documented exception.

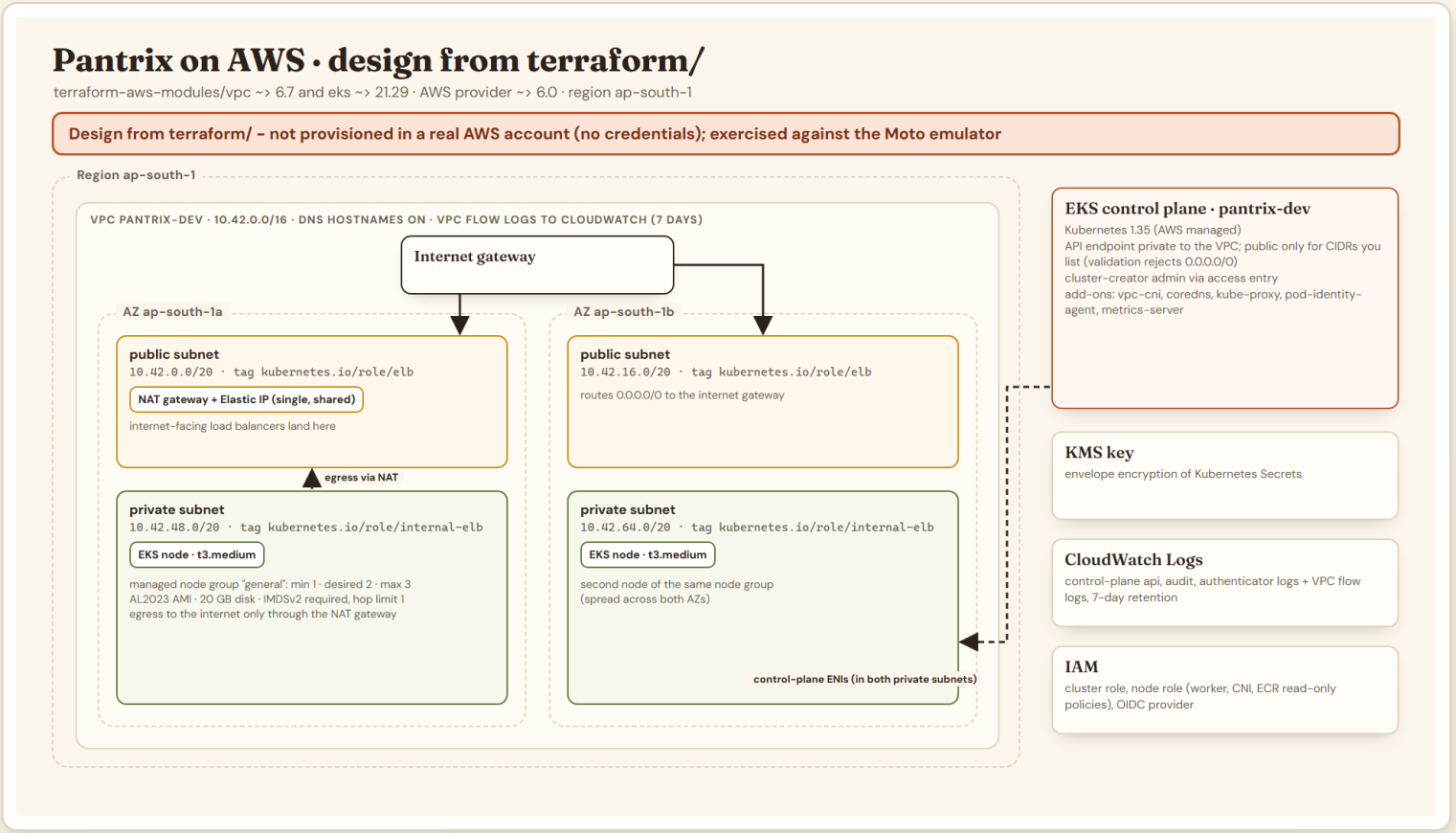
Secrets

Gitleaks over the full Git history: no leaks found. .env, state and kubeconfigs are git-ignored.

Supply chain

Every action pinned to a commit SHA. CycloneDX SBOMs: 119 components (backend), 71 (frontend). Provenance attestation on push.

Terraform: VPC and EKS as code (real AWS run pending)



62

resources in terraform plan (Moto emulator)

54 / 62

created by apply in the emulator; EKS add-on APIs missing in Moto

54

destroyed by terraform destroy (emulator)

Not provisioned in a real AWS account: this computer has no valid AWS credentials (terraform plan stops at STS InvalidClientTokenId). No AWS Console screenshots exist. fmt, init and validate pass; Trivy IaC scan passes.

Kubernetes and Helm: hardened pods, Ingress, HPA

```
dhruv-sharma@pantrix: ~/pantrix

dhruv-sharma@pantrix:~/pantrix$ kubectl get nodes -o wide
NAME                   STATUS   ROLES           AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE                         KERNEL-VERSION                              CONTAINER-RUNTIME
pantrix-control-plane   Ready    control-plane   85m   v1.37.0   172.25.0.2    <none>        Debian GNU/Linux 13 (trixie)   6.18.33.1-microsoft-standard-WSL2 (amd64)   containerd://2.3.4

dhruv-sharma@pantrix:~/pantrix$ kubectl apply -f k8s/namespace.yaml
namespace/pantrix unchanged

dhruv-sharma@pantrix:~/pantrix$ helm list -n pantrix
NAME    NAMESPACE       REVISION        UPDATED                                 STATUS          CHART           APP VERSION
pantrix pantrix         11              2026-10-12 23:37:01.8119649 +0530 IST   deployed        pantrix-1.0.0   1.0.0

dhruv-sharma@pantrix:~/pantrix$ kubectl get pods -n pantrix -o wide
NAME                                READY   STATUS    RESTARTS   AGE     IP            NODE                    NOMINATED NODE   READINESS GATES
pantrix-backend-6845997984-cw7cr    1/1     Running   0          7m50s   10.244.0.94   pantrix-control-plane   <none>           <none>
pantrix-backend-6845997984-gzn71    1/1     Running   0          7m56s   10.244.0.91   pantrix-control-plane   <none>           <none>
pantrix-frontend-796bb88c6-26m5r    1/1     Running   0          7m55s   10.244.0.93   pantrix-control-plane   <none>           <none>
pantrix-frontend-796bb88c6-pbxk8    1/1     Running   0          7m56s   10.244.0.92   pantrix-control-plane   <none>           <none>
pantrix-postgres-0                  1/1     Running   0          20m     10.244.0.56   pantrix-control-plane   <none>           <none>

Real command output captured by scripts/evidence/capture-transcripts.sh - docs/evidence/transcripts/kubernetes-resources.txt
```

kubectl get pods: 2 frontend + 2 backend + PostgreSQL, all Running

```
dhruv-sharma@pantrix: ~/pantrix

dhruv-sharma@pantrix:~/pantrix$ helm list -n pantrix
NAME    NAMESPACE       REVISION        UPDATED                                 STATUS          CHART           APP VERSION
pantrix pantrix         11              2026-10-12 23:37:01.8119649 +0530 IST   deployed        pantrix-1.0.0   1.0.0

dhruv-sharma@pantrix:~/pantrix$ helm history pantrix -n pantrix --max 6
REVISION        UPDATED                         STATUS          CHART           APP VERSION     DESCRIPTION
6               Mon Oct 12 23:30:11 2026        superseded      pantrix-1.0.0   1.0.0           Upgrade complete
7               Mon Oct 12 23:33:42 2026        failed          pantrix-1.0.0   1.0.0           Upgrade "pantrix" failed: resource Deployment/pantrix/pantrix-backend not ready. status: InPr
ogress, message: Updated: ...
8               Mon Oct 12 23:34:47 2026        superseded      pantrix-1.0.0   1.0.0           Rollback to 5
9               Mon Oct 12 23:35:44 2026        superseded      pantrix-1.0.0   1.0.0           Upgrade complete
10              Mon Oct 12 23:35:57 2026        failed          pantrix-1.0.0   1.0.0           Upgrade "pantrix" failed: resource Deployment/pantrix/pantrix-backend not ready. status: InPr
ogress, message: Updated: ...
11              Mon Oct 12 23:37:01 2026        deployed        pantrix-1.0.0   1.0.0           Rollback to 9

dhruv-sharma@pantrix:~/pantrix$ helm test pantrix -n pantrix --logs
NAME: pantrix
LAST DEPLOYED: Mon Oct 12 23:37:01 2026
NAMESPACE: pantrix
STATUS: deployed
REVISION: 11
DESCRIPTION: Rollback to 9
TEST SUITE:     pantrix-test-api
Last Started:   Mon Oct 12 23:43:41 2026
Last Completed: Mon Oct 12 23:43:47 2026
Phase:          Succeeded

POD LOGS: pantrix-test-api (api-check)
items: 17

Real command output captured by scripts/evidence/capture-transcripts.sh - docs/evidence/transcripts/helm-release.txt
```

helm list, history and helm test (passed)

```
dhruv-sharma@pantrix: ~/pantrix

dhruv-sharma@pantrix:~/pantrix$ kubectl get hpa -n pantrix
NAME               REFERENCE                     TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
pantrix-backend    Deployment/pantrix-backend    cpu: 3%/60%   2         6         2          83m
pantrix-frontend   Deployment/pantrix-frontend   cpu: 4%/70%   2         4         2          83m

# Generate load: 12 parallel traffic generators for 150 s

dhruv-sharma@pantrix:~/pantrix$ kubectl get hpa pantrix-backend -n pantrix --no-headers
pantrix-backend   Deployment/pantrix-backend   cpu: 4%/60%   2     6     2     83m

dhruv-sharma@pantrix:~/pantrix$ kubectl get hpa pantrix-backend -n pantrix --no-headers
pantrix-backend   Deployment/pantrix-backend   cpu: 397%/60%   2     6     2     84m

dhruv-sharma@pantrix:~/pantrix$ kubectl get hpa pantrix-backend -n pantrix --no-headers
pantrix-backend   Deployment/pantrix-backend   cpu: 357%/60%   2     6     6     84m

dhruv-sharma@pantrix:~/pantrix$ kubectl get hpa pantrix-backend -n pantrix --no-headers
pantrix-backend   Deployment/pantrix-backend   cpu: 203%/60%   2     6     6     84m

dhruv-sharma@pantrix:~/pantrix$ kubectl get hpa pantrix-backend -n pantrix --no-headers
pantrix-backend   Deployment/pantrix-backend   cpu: 200%/60%   2     6     6     85m

dhruv-sharma@pantrix:~/pantrix$ kubectl get hpa pantrix-backend -n pantrix --no-headers
pantrix-backend   Deployment/pantrix-backend   cpu: 196%/60%   2     6     6     85m

dhruv-sharma@pantrix:~/pantrix$ kubectl get hpa pantrix-backend -n pantrix --no-headers
pantrix-backend   Deployment/pantrix-backend   cpu: 195%/60%   2     6     6     85m

dhruv-sharma@pantrix:~/pantrix$ kubectl get hpa pantrix-backend -n pantrix --no-headers
pantrix-backend   Deployment/pantrix-backend   cpu: 143%/60%   2     6     6     86m

dhruv-sharma@pantrix:~/pantrix$ kubectl get hpa pantrix-backend -n pantrix --no-headers
pantrix-backend   Deployment/pantrix-backend   cpu: 138%/60%   2     6     6     86m

dhruv-sharma@pantrix:~/pantrix$ kubectl get hpa pantrix-backend -n pantrix --no-headers
pantrix-backend   Deployment/pantrix-backend   cpu: 38%/60%   2     6     6     86m

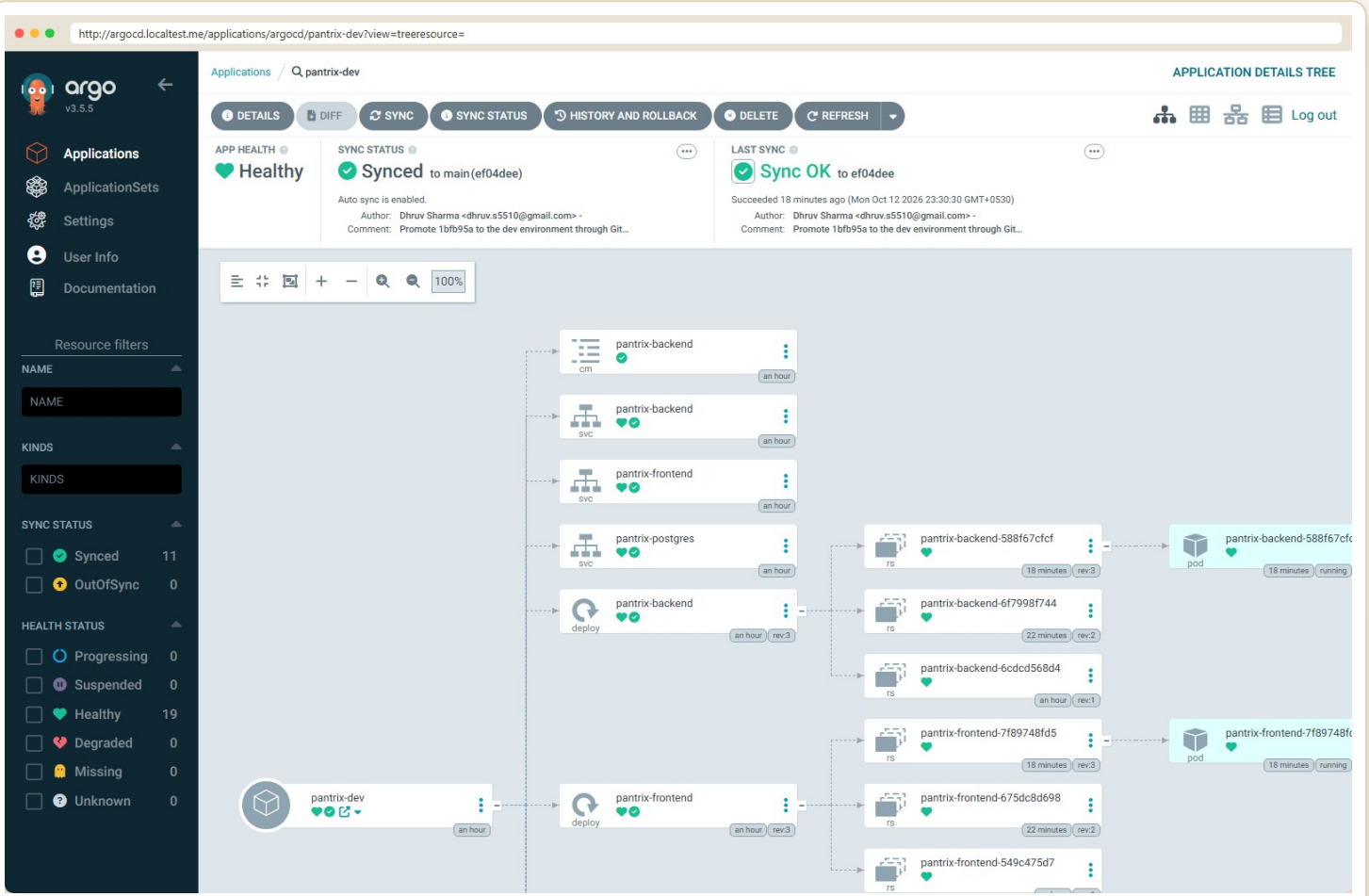
dhruv-sharma@pantrix:~/pantrix$ kubectl describe hpa pantrix-backend -n pantrix | sed -n '/^Events:/,$p'

Real command output captured by scripts/evidence/capture-transcripts.sh - docs/evidence/transcripts/hpa-scaling.txt
```

Under load the HPA went from 2 to 6 backend pods

- Ingress: / to the frontend, /api to the backend (pantrix.localtest.me)
- Restricted Pod Security: non-root, read-only root filesystem, no capabilities

GitOps with Argo CD: Git is the desired state

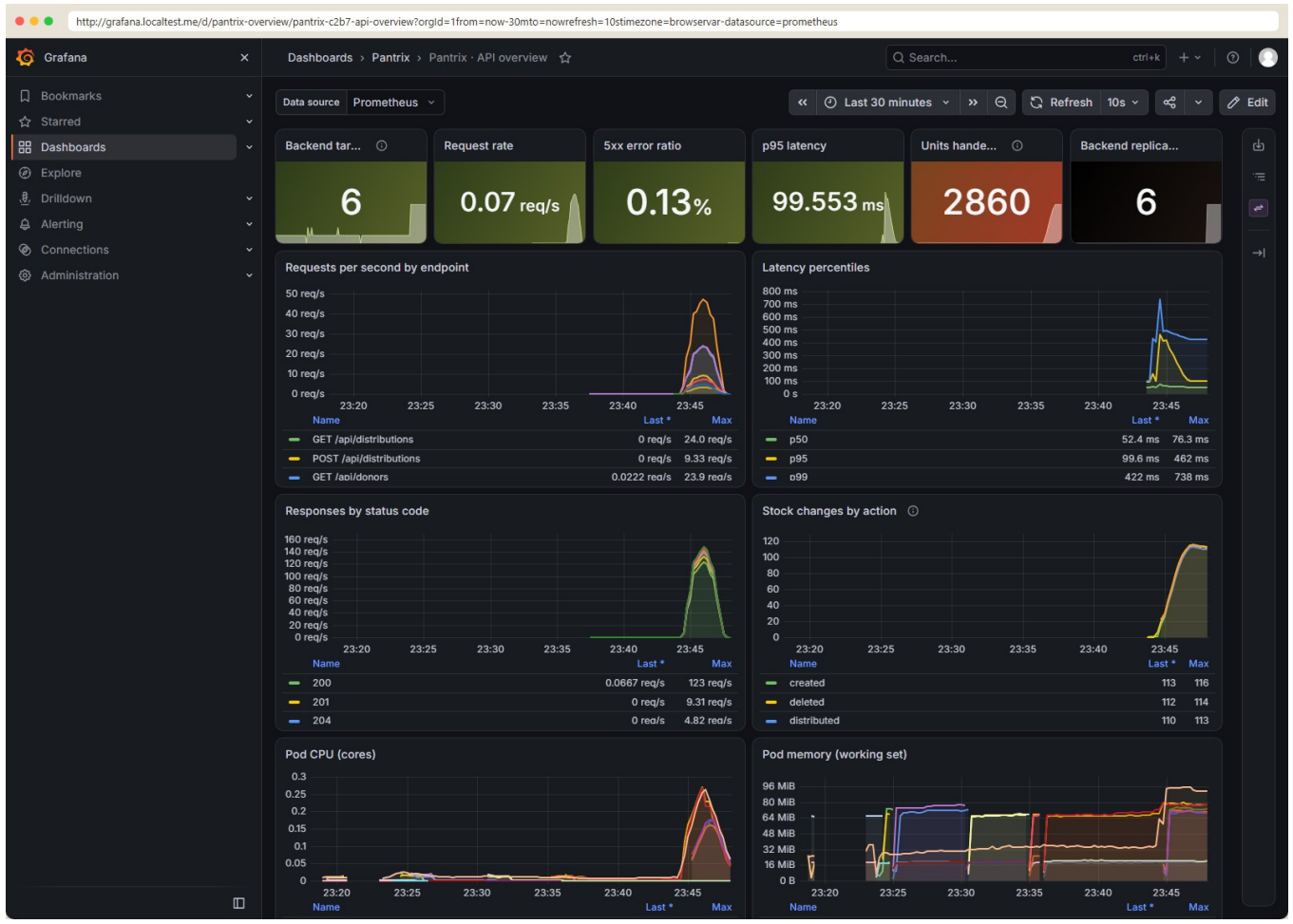


Argo CD: pantrix-dev Synced and Healthy at the promotion commit

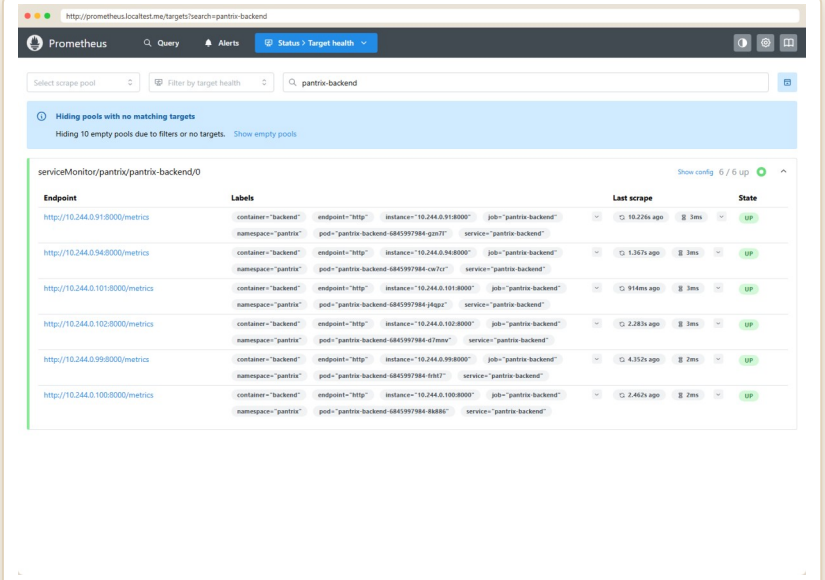
1. CI builds and scans images tagged with the commit SHA
2. A promotion commit changes image.tag in gitops/environments/dev/values.yaml
3. Argo CD sees the commit, syncs, prunes and self-heals
4. Rollback = revert the commit; history is the audit log

Until the GitHub repo exists, Argo CD watches an in-cluster git daemon; no Git remote is configured.

Observability: every pod scraped, live Grafana



Grafana: request rate, 5xx ratio, p95 latency, stock events, pod CPU/memory



Prometheus: pantrix-backend 6/6 UP

- /metrics from prometheus-fastapi-instrumentator plus Pantrix counters
- ServiceMonitor scrape every 15 s; alert rules for down, 5xx and slow p95
- Structured JSON access logs from the API

Five troubleshooting drills, run for real

Drill	Symptom	Diagnosis	Fix
1 Broken image	ImagePullBackOff	tag v9.9.9-typo was never published	deploy the Git-SHA tag
2 Broken Service	no endpoints, connection refused	selector app=shelf-frontend matches no pods	correct the selector
3 Readiness	Running but 0/1 Ready	/health 200, /ready 503: DB password wrong	use the right Secret
4 Migration	Init:Error	Alembic cannot resolve lab-postgresql	fix the database host
5 Bad release	new pods ImagePullBackOff	old pods still serve (maxUnavailable 0)	helm rollback, zero downtime

A drill found a real bug

Nginx resolved the /api upstream only at start-up, so the UI crash-looped when the backend Service did not exist yet. It now resolves at request time and the UI starts on its own (commit 1bfb95a).

What I learned, what is verified, what is pending

Key learnings

- Scanners are useful only if you fix the cause, not silence the finding
- Liveness and readiness answer different questions
- Separate build from deploy: CI publishes, Git decides what runs
- Drills find real bugs before users do

Verified here

- App, CRUD, 41 tests, Compose stack, non-root images
- Trivy, Gitleaks, SBOMs, IaC scan
- Helm on kind, Ingress, HPA 2 to 6, Argo CD Synced
- Prometheus 6/6 UP, Grafana with live traffic

Pending (external access)

- First GitHub Actions run and GHCR packages (after the repo is pushed)
- Real AWS VPC + EKS apply, Console proof and destroy (needs AWS credentials and approval)

Thank you · Dhruv Sharma · github.com/spiritsfuse